



UNIVERSITY OF MINES AND TECHNOLOGY
TARKWA

DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS

**BUILDING A THREAT INTELLIGENCE PLATFORM
(TIP)**
**INTEGRATION FOR A SIMULATED SECURITY
OPERATIONS CENTER (SOC)**

NAME:	IBRAHIM ABDUL AZIZ
INDEX NUMBER:	FCM.41.018.151.23
LEVEL:	300 BSc CYBERSECURITY
COURSE CODE:	CY376
COURSE:	NETWORK MONITORING, SECURITY AND AUDITING
LECTURER:	MR. FREDRICK BRONI
TEAM:	BLUE TEAM
DATE:	16TH AUGUST 2026
REPOSITORY:	github.com/aibrahim8523/cy376-tip-soc-project

Abstract

Modern Security Operations Centres (SOCs) need more than raw alerts: analysts require immediate context about whether an IP address or file hash is already associated with malicious activity. This project implemented and validated a consolidated Threat Intelligence Platform-SIEM prototype using MISP and Wazuh on one Ubuntu Server. The server was reachable at 192.168.18.130 on a VMware NAT network; MISP was delivered through Docker on HTTPS port 443, while the Wazuh Dashboard used HTTPS port 8443. Kali Linux at 192.168.18.128 generated controlled attack traffic.

A MISP event named “Simulated Malware Campaign – Lab Exercise” stored three indicators: the Kali address 192.168.18.128, the EICAR SHA-256 hash, and malicious-lab-test.local. Wazuh detected Kali-originated SSH activity through custom rule 100002 at level 12, and detected the EICAR file through real-time File Integrity Monitoring rule 554. A custom Python integration automatically extracted the relevant IP address or SHA-256 hash, queried the MISP restSearch API, and recorded a successful HTTP 200 response with one matching attribute in each scenario. The prototype therefore demonstrates practical, automated enrichment for both network and file indicators using open-source tools.

Keywords: MISP, Wazuh, SIEM, Threat Intelligence Platform, SOC, IOC, EICAR, File Integrity Monitoring, MITRE ATT&CK;

Table of Contents

Abstract	2
Table of Contents	3
1. Introduction	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Aim	5
1.4 Objectives	5
1.5 Scope	5
2. Literature and Tooling Review	6
2.1 Cyber Threat Intelligence and IOCs	6
2.2 STIX, TAXII and TLP	6
2.3 Blue-Team Frameworks	6
2.4 Selected Tools	6
2.5 Related Work	6
2.6 Tool and Code Provenance	6
3. Methodology	8
3.1 Research Design	8
3.2 Implemented Lab Environment	8
3.3 Experimental Scenarios	9
3.4 Validation Criteria	9
3.5 Evidence Collection and Handling	9
4. Implementation	10
4.1 Network and Resource Preparation	10
4.2 Wazuh Deployment	10
4.3 Docker and MISP Deployment	10
4.4 MISP Configuration	10
4.5 MISP Attributes	11
4.6 Custom MISP Integration	13
4.7 Custom Rules and FIM	13
4.8 GitHub Repository Organisation	14
4.9 Troubleshooting and Stabilisation	14
4.9.1 Resource Constraints	15
4.9.2 Resilient Downloads	15
4.9.3 Initial MISP Environment and URL Correction	16
4.9.4 Integration Argument Discovery	16

4.9.5 FIM Database Repair	16
4.10 Operational Verification	17
5. Results and Findings	18
5.1 Docker and MISP Results	18
5.2 Known-Bad IP Detection	18
5.3 MISP IP Enrichment	19
5.4 EICAR File Detection	19
5.5 SSH Brute-Force Detection	20
5.6 Results Summary	20
5.7 Evidence Traceability Matrix	21
5.8 Measurement and Observations	21
5.9 Screenshot Interpretation	21
6. Analysis and Recommendations	22
6.1 Interpretation	22
6.2 MITRE ATT&CK; Mapping	22
6.3 Limitations	22
6.4 Recommendations	22
6.5 Comparison with a SIEM-Only Workflow	23
6.6 Operational Lessons	23
6.7 Recommended Production Architecture	23
7. Conclusion	24
8. References	25
9. Appendices	26
Appendix A — Key Commands	26
Appendix B — Deployment Evidence	26
Appendix C — Security and Reproducibility Notes	26
Appendix D — GitHub Submission Checklist	27
Appendix E — Presentation Evidence Plan	27
Appendix F — Custom Integration Logic	27
Appendix G — Wazuh Rule and Integration Excerpts	28
Appendix H — Detailed Troubleshooting Commands	28
Appendix I — Real Screenshot Index	28
Appendix J — README Outline	29
Appendix K — Ten-Minute Presentation Outline	29

1. Introduction

1.1 Background

Security Operations Centres collect endpoint, network, authentication and application logs and use SIEM platforms to convert those observations into alerts. A SIEM alert may identify a suspicious address or hash but usually does not explain whether the indicator is already known to the security community. Analysts must then perform separate lookups, increasing mean time to triage and the possibility of inconsistent decisions.

Cyber Threat Intelligence provides context for those observations. MISP stores indicators of compromise (IOCs), their categories, event relationships and distribution controls. Wazuh provides log analysis, file-integrity monitoring, rule correlation and dashboard visualisation. Integrating the two allows a Wazuh alert to trigger an immediate MISP lookup.

1.2 Problem Statement

Rule-only detections provide limited context and require analysts to manually validate IP addresses, domains and hashes. The project addresses this problem through a repeatable lab prototype in which Wazuh automatically queries MISP whenever selected network or FIM rules fire.

1.3 Aim

To design, implement and validate a MISP-Wazuh integration that enriches Wazuh detections with matching threat-intelligence indicators in a controlled SOC lab.

1.4 Objectives

- Deploy MISP and Wazuh together on an Ubuntu Server while avoiding HTTPS port conflicts.
- Create a MISP event containing an attacker IP, EICAR SHA-256 hash and test domain.
- Detect Kali SSH and scanning activity through Wazuh rules.
- Detect EICAR through real-time Wazuh File Integrity Monitoring.
- Automatically query the MISP API for both IP and hash indicators.
- Capture real implementation and validation evidence for the final report.

1.5 Scope

The prototype uses one Ubuntu Server as the consolidated SOC host and local monitored endpoint, one Kali VM as the attacker, and a Windows host for administration. Automated containment and enterprise-scale performance testing remain outside scope.

2. Literature and Tooling Review

2.1 Cyber Threat Intelligence and IOCs

Cyber threat intelligence is evidence-based knowledge used to support defensive decisions. Operational IOCs include IP addresses, domain names, URLs and cryptographic file hashes. Their value increases when they are linked to events, confidence, categories and observed activity rather than treated as isolated strings [1], [9].

2.2 STIX, TAXII and TLP

STIX provides a structured language for threat information, TAXII supports automated exchange, and the Traffic Light Protocol communicates sharing restrictions [5], [6], [10]. MISP supports standards-based exchange while also exposing a native REST API suited to local integrations.

2.3 Blue-Team Frameworks

MITRE ATT&CK; provides technique identifiers for adversary behaviour. The SSH password-guessing scenario maps to T1110.001 and T1021.004, while the higher-level brute-force correlation maps to T1110. File-integrity alerts support detection of payload placement and modification [7].

2.4 Selected Tools

- MISP 2.5.44: TIP, event/attribute management, feeds and REST API.
- Wazuh 4.9.2: manager, indexer, dashboard, Filebeat, rules and FIM.
- Docker Compose: MISP core, modules, MariaDB, Valkey/Redis and SMTP services.
- Kali Linux: Nmap and SSH activity from 192.168.18.128.
- EICAR: harmless standard test file with a known SHA-256 hash.

2.5 Related Work

Existing Wazuh and MISP documentation explains individual configuration components. This project contributes a compact implementation record showing port coexistence, memory constraints, automatic indicator extraction, real-time detection and real evidence from the completed lab.

2.6 Tool and Code Provenance

MISP, Wazuh, Docker, Kali Linux and EICAR are third-party open-source or publicly documented tools and are cited in Section 8. The MISP deployment uses the official misp-docker Compose template. The Wazuh installation uses the official installation assistant. The custom-misp wrapper and Python logic were developed and adapted in the lab to process the Wazuh integration argument format, query the MISP API and record matches. No third-party implementation is presented as original student code.

All attack activity was generated inside the student-controlled VMware network. The screenshots in this report are real captures from that

environment. Credentials and the API key are redacted in the guideline-compliant copy, and the exposed lab key must be revoked before the repository is made public.

3. Methodology

3.1 Research Design

A constructive design was used: build the consolidated SOC host, create known test indicators, generate controlled stimuli, observe Wazuh rules, query MISP, and compare the returned attributes with the original indicators. Success required healthy services, repeatable alerts and HTTP 200 responses from MISP.

3.2 Implemented Lab Environment

System	Role	Resources / OS	Address / Port
Windows host	Browser and SSH administration	Windows host system	VMware NAT access
Ubuntu Server	MISP, Wazuh and monitored local endpoint	Ubuntu 22.04; 4 GB RAM; 2 GB swap; 34 GB LVM	192.168.18.130; MISP 443; Wazuh 8443
Kali Linux	Threat simulator	Kali rolling	192.168.18.128

The consolidated design was selected because of host storage limits. MISP and Wazuh perform different logical roles but share the same Ubuntu Server. Port 8443 was assigned to Wazuh because MISP uses port 443.

SIMULATED SOC — IMPLEMENTED ARCHITECTURE

VMware NAT network · 192.168.18.0/24

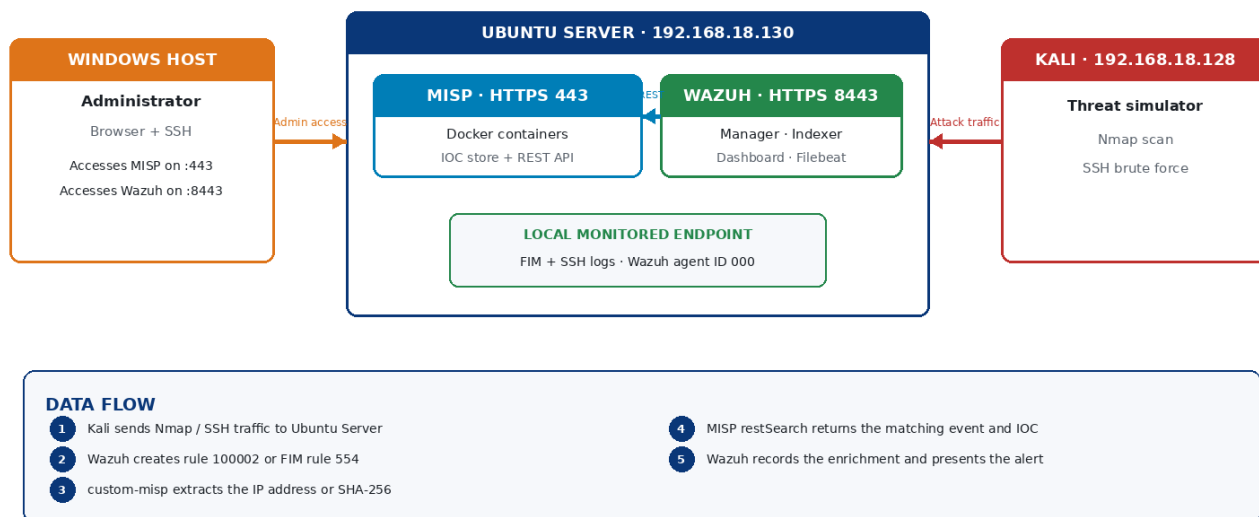


Figure 1 — Implemented logical architecture of the simulated SOC.

3.3 Experimental Scenarios

Scenario	Stimulus	Expected Wazuh Detection	MISP Indicator
S1 — Known-bad IP	Nmap and SSH from Kali	Rule 100002, level 12	ip-dst 192.168.18.128
S2 — EICAR	Create 68-byte EICAR file	FIM rule 554, level 5	sha256 275a021b...651fd0f
S3 — Brute force	Repeated failed SSH logins	Rules 5710, 5760 and 2502	Behavioural detection; IP available for lookup

AUTOMATED MISP-WAZUH ENRICHMENT FLOW

Implemented and validated for an IP indicator and the EICAR SHA-256

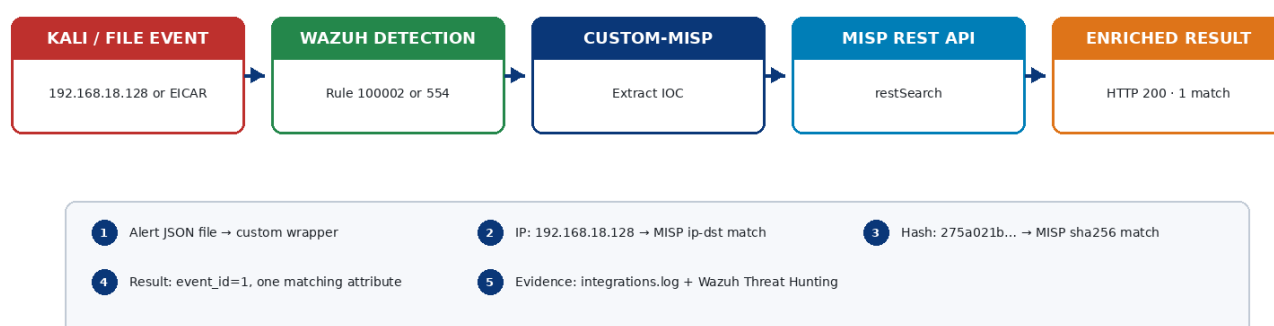


Figure 2 — Automated Wazuh-to-MISP enrichment flow.

3.4 Validation Criteria

- MISP and Wazuh interfaces load from the Windows host.
- All Wazuh and Docker services remain healthy together.
- Kali activity produces a Wazuh security event.
- EICAR produces a real-time syscheck event and the expected hash.
- The custom integration receives HTTP 200 and exactly one matching attribute.

3.5 Evidence Collection and Handling

Evidence was captured immediately after each milestone. Browser and terminal screenshots were cropped to the relevant application area, numbered, captioned and discussed in the report. The test addresses are private VMware NAT addresses. The MISP API key has been hidden in this version because the submission guidelines prohibit credentials in the printed report and Git history.

4. Implementation

4.1 Network and Resource Preparation

The Ubuntu Server used VMware NAT address 192.168.18.130. Kali used 192.168.18.128. Wazuh originally experienced out-of-memory termination when running alongside MISP. A 2 GB swap file was added and the Wazuh indexer was restarted successfully. Two virtual disks were combined through LVM, providing approximately 34 GB usable storage.

4.2 Wazuh Deployment

```
sudo bash wazuh-install.sh -a -o -p 8443
# Dashboard: https://192.168.18.130:8443
```

The all-in-one installer deployed Wazuh Manager, Indexer, Dashboard and Filebeat. The Dashboard was moved to port 8443 so that MISP could retain HTTPS port 443. All four services were enabled and validated with systemctl.

4.3 Docker and MISP Deployment

Docker and containerd were installed and validated with the hello-world image. The MISP Compose stack used MariaDB, Valkey/Redis, SMTP, misp-modules and misp-core. Large image downloads were resumed until complete. The resulting containers reported healthy states.

```
ibrahim@ubuntuserver:~$ sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
```

Figure 3 — Real Docker hello-world validation output.

NAME	IMAGE	COMMAND	SERVICE
TATUS	PORTS		
misp-docker-db-1	mariadb:10.11	"docker-entrypoint.s..."	db
p 6 minutes (healthy)	3306/tcp		
misp-docker-mail-1	ghcr.io/egos-tech/smtp:1.1.3	"/bin/entrypoint.sh ..."	mail
p 6 minutes	25/tcp		
misp-docker-misp-core-1	ghcr.io/misp/misp-docker/misp-core:latest	"/entrypoint.sh"	misp-core
p 34 seconds (healthy)	0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp		
misp-docker-misp-modules-1	ghcr.io/misp/misp-docker/misp-modules:latest	"/usr/local/bin/misp..."	misp-modules
p 6 minutes (healthy)			
misp-docker-redis-1	valkey/valkey:7.2	"docker-entrypoint.s..."	redis
p 6 minutes (healthy)	6379/tcp		

Figure 4 — Real MISP Docker container status showing the deployed services.

4.4 MISP Configuration

MISP was configured with BASE_URL https://192.168.18.130 and administrator account admin@admin.test. The platform served HTTPS on port 443. An event named “Simulated Malware Campaign – Lab Exercise” was created with high threat level and three curated indicators.

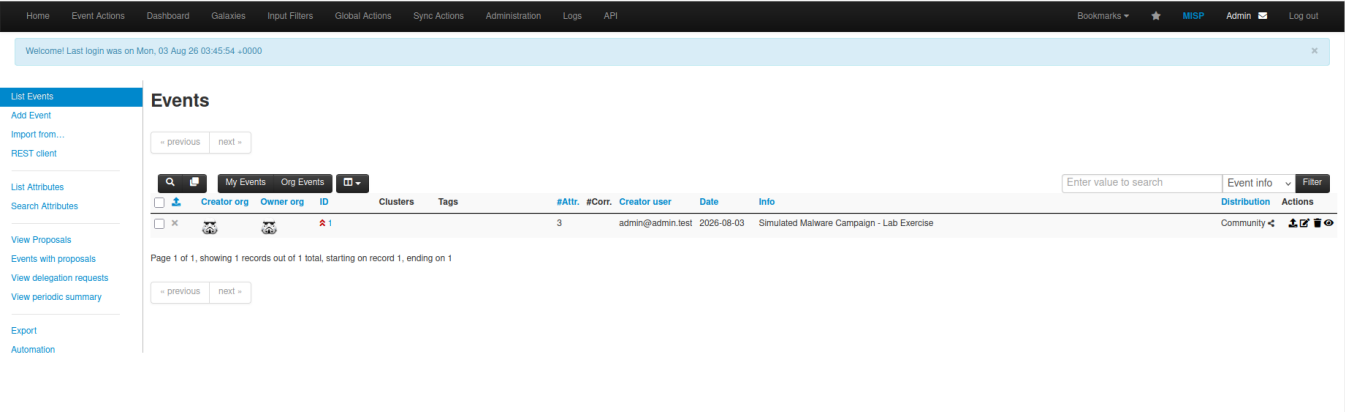


Figure 5 — Real MISP Events page showing the simulated malware campaign.

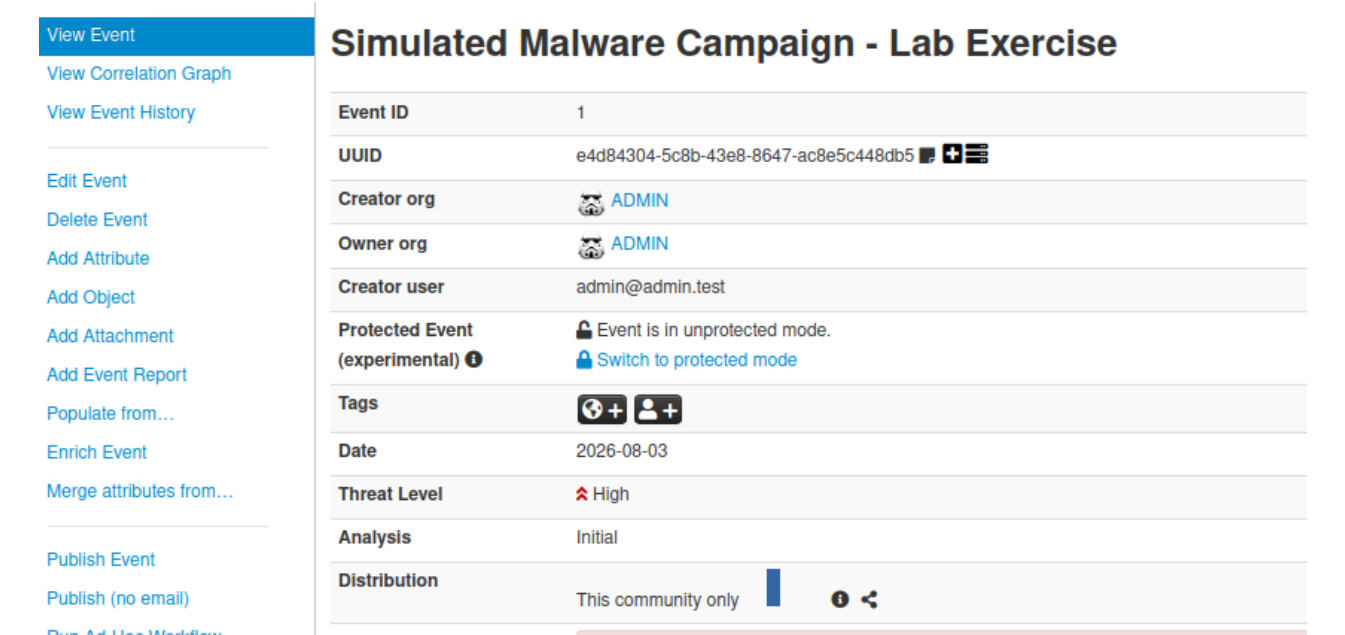


Figure 6 — Real MISP event details for Event ID 1.

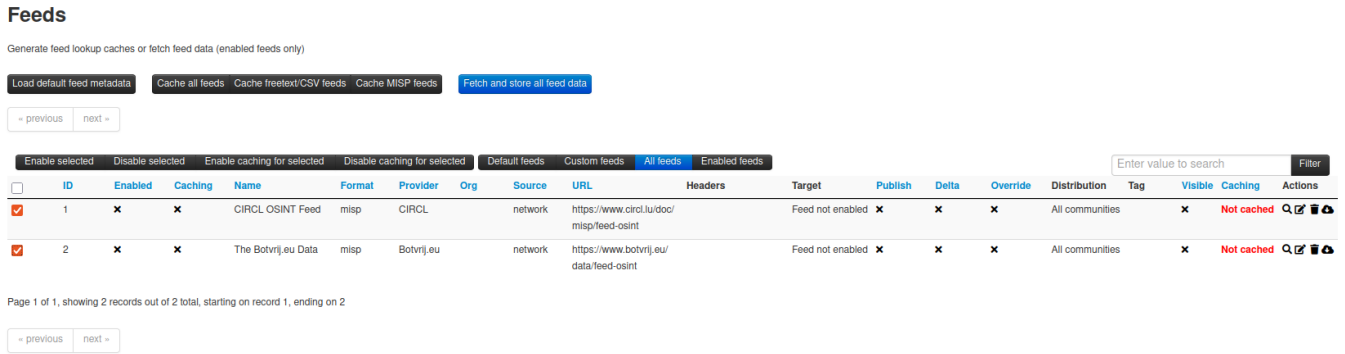


Figure 7 — Real MISP feeds page with available feed metadata.

4.5 MISP Attributes

The event registered Kali as an ip-dst indicator, the EICAR SHA-256 as a file indicator, and malicious-lab-test.local as a domain indicator. The captured forms below are the actual MISP interface used during the lab.

Add Attribute
✕

Category ⓘ
Type ⓘ

Network activity
ip-dst

Distribution ⓘ

Inherit event

Value

192.168.18.128

Contextual Comment

Simulated known-bad C2 IP

☐ Batch import ⓘ
☐ For Intrusion Detection System
☐ Disable Correlation

First seen date
Last seen date

Figure 8 — Real MISP IP attribute: Kali 192.168.18.128.

Add Attribute
✕

Category ⓘ
Type ⓘ

Artifacts dropped
sha256

Distribution ⓘ

Inherit event

Value

275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f

Contextual Comment

EICAR test file - harmless antivirus test string

Figure 9 — Real MISP SHA-256 attribute for EICAR.

Add Attribute

Category ⓘ Type ⓘ

Network activity domain

Distribution ⓘ

Inherit event

Value

malicious-lab-test.local

Contextual Comment

Test domain

Figure 10 — Real MISP domain attribute.

4.6 Custom MISP Integration

Wazuh was configured with a custom-misp integration referencing the MISP restSearch endpoint. A wrapper receives the temporary Wazuh alert file, API key and URL. The Python component extracts either data.srcip/data.dstip or syscheck.sha256_after, queries MISP and records the result in integrations.log and a structured enrichment log.

```
<!-- MISP integration -->
<integration>
  <name>custom-misp</name>
  <hook_url>https://192.168.18.130/attributes/restSearch</hook_url>
  <api_key>LJG [LAB API KEY REDACTED] </api_key>
  <alert_format>json</alert_format>
</integration><ossec_config>
```

Figure 11 — Real Wazuh custom-misp integration configuration.

```
<integration>
<name>custom-misp</name>
<hook_url>https://192.168.18.130/attributes/restSearch</hook_url>
<api_key>LAB_API_KEY</api_key>
<alert_format>json</alert_format>
<rule_id>2502,5760,5763,5503,5715,100002,554</rule_id>
</integration>
```

4.7 Custom Rules and FIM

Custom rule 100002 assigns level 12 when logs contain the MISP-registered Kali IP. Real-time FIM was enabled for /home/ibrahim/eicar-test. An existing 100,000-file limit was resolved by resetting the stale FIM database and temporarily restricting monitoring to the dedicated test directory.

```
<rule id="100002" level="12">
<match>192.168.18.128</match>
<description>Connection from MISP-registered known-bad IP (Kali)</description>
</rule>

<directories realtime="yes" check_all="yes">/home/ibrahim/eicar-test</directories>
```

4.8 GitHub Repository Organisation

The repository URL is printed on the cover page. The recommended structure places the report in `report/`, configuration excerpts in `configs/`, integration and helper code in `scripts/`, documentation in `docs/`, and original screenshots and log evidence in `evidence/`. A root README describes the project, tools, reproduction steps and key results. A `.gitignore` excludes `.env` files, API keys, certificates, VM disks and other secrets.

```
cy376-tip-soc-project/
├── README.md
├── .gitignore
├── report/
├── configs/
├── scripts/
├── docs/
└── evidence/
```

4.9 Troubleshooting and Stabilisation

The final prototype was reached through a sequence of measurable troubleshooting activities. These were not treated as incidental errors: each issue was diagnosed from service status, logs, package state or resource measurements and then validated after correction. Table 3 records the principal problems and their resolutions.

Problem	Observed evidence	Resolution	Validation
Wazuh indexer package download failure	850 MB download returned timeout / short-read errors	Used resumable wget download and reran the installer	Indexer installed, cluster initialised and service active
Disk exhaustion	VM filesystem lacked space during package extraction	Combined two 18 GB virtual disks through LVM, yielding about 34 GB	df -h reported sufficient free space
Indexer OOM termination	systemd reported Result: oom-kill	Added 2 GB swap and retained a reduced lab workload	Indexer remained active beside MISP
Docker/containerd failure	containerd failed with stale state under /var/lib/containerd	Stopped services, cleared stale containerd state and restarted in order	hello-world completed successfully
MISP image pull interruptions	TLS timeout and unexpected EOF during large pulls	Repeated resumable pulls until layers completed	All Compose containers became healthy
MISP redirect to old address	MISP redirected 192.168.18.130 to 192.168.100.10	Corrected BASE_URL, MISP_BASEURL and config.php values	HTTP 302 redirected to the correct login URL

Problem	Observed evidence	Resolution	Validation
Integration not executed	integrator reported custom-misp file not found	Created executable wrapper matching the integration name	integrator logged Enabling integration
Invalid alert JSON	Wazuh passed a temporary alert-file path rather than raw JSON	Updated Python component to open argv[1] and read key/URL from argv[2]/argv[3]	Controlled and automatic MISP queries returned HTTP 200
FIM file limit	Rule 233 reported 100000 monitored files	Reset stale FIM DB and temporarily limited monitoring to eicar-test	Rule 554 detected test and EICAR files in real time

4.9.1 Resource Constraints

MISP and Wazuh are normally separated in production. Consolidating them on a 4 GB VM placed the Java-based indexer, MariaDB, MISP core, Redis/Valkey and other services under memory pressure. The indexer was killed by the Linux OOM mechanism. A 2 GB swap file provided sufficient virtual memory for the prototype. Final verification showed all Wazuh services, Docker and containerd active, although most swap space was in use; this confirms the design is suitable for demonstration but not for production capacity.

4.9.2 Resilient Downloads

The 850 MB Wazuh indexer package and large MISP images repeatedly failed on the available network. Resumable download methods were used rather than restarting the complete deployment. Docker retained completed layers, while `wget -c` resumed the Wazuh package. This reduced bandwidth waste and preserved evidence of a realistic constrained-network deployment.

```

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Figure 12 — Complete real Docker hello-world output used to validate the repaired daemon.

4.9.3 Initial MISP Environment and URL Correction

The initial MISP environment used an address from the earlier host-only design. After the operational network changed to VMware NAT, the server remained reachable at 192.168.18.130 but MISP redirected users to the old address. The final correction updated both environment variables and the generated MISP configuration. Figure 13 is retained as troubleshooting evidence; the credential is redacted and the obsolete address is explicitly identified as an intermediate state.

```
# CORE_RUNNING_TAG=latest
# MODULES_RUNNING_TAG=latest
# GUARD_RUNNING_TAG=latest

# optional: use a registry mirror for docker images (include trailing slash, e.g., internal.mirror.)
# REGISTRY_MIRROR_URL=

# Email/username for user #1, defaults to MISP's default (admin@admin.test)
ADMIN_EMAIL=admin@admin.test
# name of org #1, default to MISP's default (ORGNAME)
ADMIN_ORG=
# uuid of org #1, defaults to an automatically generated one
ADMIN_ORG_UUID=
# defaults to an automatically generated one
ADMIN_KEY=
# defaults [PASSWORD REDACTED]
ADMIN_PASSWORD=1111
# Prevent MISP Initialization from writing ADMIN_KEY and ADMIN_PASSWORD in plaintext
# Recommend uncommenting / setting to true in production or kubernetes environments where output is
#DISABLE_PRINTING_PLAINTEXT_CREDENTIALS=true
# defaults to 'passphrase'
GPG_PASSPHRASE=
# defaults to 1 (the admin user)
CRON_USER_ID=
# defaults to 'https://localhost'
# note: if you are exposing MISP on a non-standard port (i.e., the port is part of the URL you would use)
BASE_URL=https://192.168.100.10
# defaults to 80 and 443, don't forget to update the base url if not the defaults one
# CORE_HTTP_PORT=
# CORE_HTTPS_PORT=
# store settings in db except those that must stay in config.php. true/false, defaults to false
ENABLE_DB_SETTINGS=
```

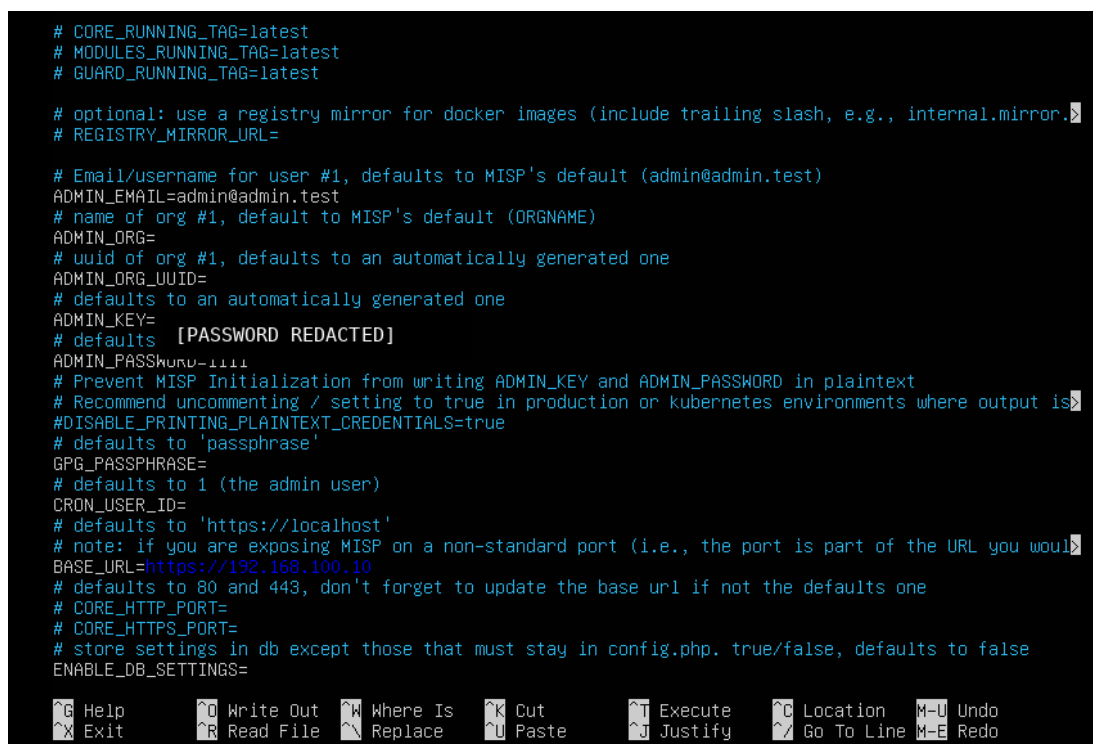


Figure 13 — Real initial MISP environment configuration before the base-URL correction (credential redacted).

4.9.4 Integration Argument Discovery

Wazuh integratord invokes a custom integration with a temporary alert file followed by the API key, hook URL, alert level and group information. The wrapper was instrumented to record its arguments. The Python component was then corrected to open the temporary JSON file instead of parsing the file path itself. This eliminated the invalid-alert-JSON error and enabled repeatable automatic queries.

4.9.5 FIM Database Repair

The FIM dashboard initially contained only rule 233 warnings. Resetting the database while Wazuh was fully stopped removed the stale 59 MB baseline. Broad default paths were temporarily removed for the prototype, leaving /home/ibrahim/eicar-test under real-time monitoring. The rebuilt baseline completed immediately, and both a normal file and EICAR generated rule 554 events.

4.10 Operational Verification

The final health check returned active for wazuh-manager, wazuh-indexer, wazuh-dashboard, filebeat, docker and containerd. Docker Compose reported healthy for MariaDB, misp-core, misp-modules and Valkey/Redis. This check was performed after both indicator scenarios, demonstrating that the platform remained operational under the final configuration.

5. Results and Findings

5.1 Docker and MISP Results

The Docker engine executed correctly, images were downloaded, and the MISP Compose services reached healthy states. Event ID 1 was available in the MISP interface with the three required indicators.

```
WARN[0000] The "GUARD_COMMIT" variable is not set. Defaulting to a blank string.
WARN[0000] The "SES_USER" variable is not set. Defaulting to a blank string.
WARN[0000] The "SES_PASSWORD" variable is not set. Defaulting to a blank string.
WARN[0000] The "SES_REGION" variable is not set. Defaulting to a blank string.
WARN[0000] The "SES_PORT" variable is not set. Defaulting to a blank string.
[+] Running 0/5
  ⚡ misp-core Pulling      27.9s
  ⚡ misp-modules Pulling   27.9s
  ⚡ mail Pulling           27.9s
  ⚡ redis Pulling          27.9s
  ⚡ db Pulling             27.9s
```

Figure 12 — Real Docker image pull output.

```
431d88c5db17 Pull complete      3.1s
6810ddcdcd9 Pull complete      2308.9s
b9c877046c76 Pull complete     2498.2s
[+] Running 3/3
  ⚡ Network misp-docker_default Created      0.1s
[+] Running 4/8docker_misp_guard_ca Created      0.0s
[+] Running 9/9-docker_default Created      0.1s
  ⚡ Network misp-docker_default Created      0.1s
  ⚡ Volume misp-docker_misp_guard_ca Created    0.0s
  ⚡ Volume misp-docker_cache_data Created      0.0s
  ⚡ Volume misp-docker_mysql_data Created      0.0s
  ⚡ Container misp-docker-misp-modules-1 Healthy 10.1s
  ⚡ Container misp-docker-db-1 Healthy      12.6s
  ⚡ Container misp-docker-mail-1 Started      2.4s
  ⚡ Container misp-docker-redis-1 Healthy      8.1s
  ⚡ Container misp-docker-misp-core-1 Started   12.3s
ibrahim@ubuntuuserver:~/misp-docker$
```

Figure 13 — Real Compose network, volumes and containers created.

NAME	SERVICE	CREATED	IMAGE	STATUS	PORTS	COMMAND
misp-docker-db-1	db	3 minutes ago	mariadb:10.11	Up 3 minutes (healthy)	3306/tcp	"docker-entrypoint.s..."
misp-docker-mail-1	mail	3 minutes ago	ghcr.io/egos-tech/smtp:1.1.3	Up 3 minutes	25/tcp	"/bin/entrypoint.sh ..."
misp-docker-misp-core-1	misp-core	3 minutes ago	ghcr.io/misp/misp-docker/misp-core:latest	Up 3 minutes (healthy)	0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp	"/entrypoint.sh"
misp-docker-misp-modules-1	misp-modules	3 minutes ago	ghcr.io/misp/misp-docker/misp-modules:latest	Up 3 minutes (healthy)		"/usr/local/bin/misp..."
misp-docker-redis-1	redis	3 minutes ago	valkey/valkey:7.2	Up 3 minutes (healthy)	6379/tcp	"docker-entrypoint.s..."

ibrahim@ubuntuuserver:~/misp-docker\$

Figure 14 — Real locally available container images.

5.2 Known-Bad IP Detection

Kali performed a SYN scan of 192.168.18.130. The scan confirmed SSH on port 22, MISP on ports 80/443 and Wazuh on port 8443. Subsequent SSH attempts generated authentication events containing Kali address 192.168.18.128. Custom rule 100002 raised these events to level 12.

```
(kali@kali)-[~]
$ nmap -sS 192.168.18.130
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-15 11:38 EDT
Nmap scan report for 192.168.18.130
Host is up (0.0012s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   open  https
8443/tcp  open  https-alt
MAC Address: 00:0C:29:32:02:42 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.74 seconds
```

Figure 15 — Real Kali Nmap scan against the consolidated SOC server.

	agent.name	rule.description	rule.level	rule.id
16:01:46.830	ubuntuserver	Connection from MISP-registere...	12	100002
15:59:20.674	ubuntuserver	Connection from MISP-registere...	12	100002
15:58:12.569	ubuntuserver	Connection from MISP-registere...	12	100002

Figure 16 — Real Wazuh Threat Hunting results for rule 100002, level 12.

5.3 MISP IP Enrichment

The custom integration queried the MISP API with 192.168.18.128. MISP returned HTTP 200 and one matching attribute from Event ID 1. This result verifies that Wazuh can use the local TIP to validate a network indicator.

```
INFO: Querying MISP for value=192.168.18.128
INFO: MISP response code 200
INFO: MISP returned 1 matching attribute(s)
INFO: Enrichment attached: event_id=1 value=192.168.18.128
```

Figure 17 — Real controlled MISP integration query and one-attribute match.

```
custom-misp: INFO: Rule 100002; querying MISP for value=192.168.18.128
custom-misp: INFO: MISP response code=200
custom-misp: INFO: MISP returned 1 matching attribute(s)
custom-misp: MATCH: Enrichment attached: event_id=1, type=ip-dst, value=192.168.18.128
```

5.4 EICAR File Detection

After resetting the stale FIM database, Wazuh completed a new baseline and detected a normal test file and the EICAR file in real time. Rule 554 reported the EICAR file as added, with size 68 bytes and the exact SHA-256 registered in MISP.

```
$ sha256sum eicar.com
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             %             Dload  Upload   Total   Spent    Left   Speed
  0     0     0     0     0     0      0      0  --:--:--  --:--:--    0     0     0
  0     0     0     0     0     0      0      0  --:--:--  0:00:01 100    68 100    68
100   68     0     0    24     0    0:00:02  0:00:02 --:--:--   24
-rw-rw-r-- 1 ibrahim ibrahim 68 Aug 15 23:14 eicar.com
275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f eicar.com
```

Figure 18 — Real EICAR download, 68-byte size and SHA-256 verification.

Information			
ID 554	Level 5	File 0015-ossec_rules.xml	Path ruleset/rules
Groups syscheck, syscheck_entry_added, syscheck_file, ossec			
Details			
Category ossec	Decoded_as syscheck_new_entry		

Figure 19 — Real Wazuh rule 554 information for the EICAR FIM alert.

The integration was extended to read syscheck.sha256_after. When rule 554 fired again, custom-misp automatically queried MISP with the EICAR SHA-256, received HTTP 200 and returned one matching sha256 attribute from Event ID 1.

```

custom-misp: INFO: Rule 554; querying MISP for sha256
value=275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f
custom-misp: INFO: MISP response code=200
custom-misp: INFO: MISP returned 1 matching attribute(s)
custom-misp: MATCH: Enrichment attached: event_id=1, type=sha256,
value=275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f

```

5.5 SSH Brute-Force Detection

Repeated failed SSH attempts produced native Wazuh rules 5710 and 5760 and correlation rule 2502. The events map to MITRE ATT&CK; T1110 and demonstrate that behavioural rules continue to operate alongside threat-intelligence matching.

5.6 Results Summary

Scenario	Wazuh result	MISP result	Evidence
Known-bad Kali IP	Rule 100002; level 12	HTTP 200; one ip-dst match; event 1	Figures 15-17
EICAR file	Rule 554; level 5; realtime FIM	HTTP 200; one sha256 match; event 1	Figures 18-19 + log excerpt
SSH brute force	Rules 5710, 5760 and 2502	Behavioural detection; IP available to integration	Threat Hunting / alert log

5.7 Evidence Traceability Matrix

Requirement	Evidence source	Key observable	Interpretation
TIP deployment	Figures 4-10 and Docker appendix	MISP Event ID 1, feeds, IOC forms, healthy containers	Threat-intelligence platform operational
SIEM deployment	Service checks and Wazuh screenshots	All central services active; dashboard on 8443	SIEM operational
Known-bad network indicator	Figures 15-17	Rule 100002, level 12, Kali IP 192.168.18.128	Local detection and MISP IP correlation
File indicator	Figures 18-19 and integration log	Rule 554, 68 bytes, matching SHA-256	Real-time FIM and MISP hash correlation
Behavioural detection	Alert log / Threat Hunting	Rules 5710, 5760 and 2502	SSH password-guessing detection
Reproducibility	Appendix commands and repository plan	Addresses, ports, rules and paths recorded	Lab can be rebuilt and defended in interview

5.8 Measurement and Observations

The local REST lookups completed with HTTP 200 and one returned attribute in both deterministic scenarios. The IP test matched event ID 1 and type ip-dst; the file test matched event ID 1 and type sha256. The Wazuh evidence shows rule 100002 at level 12 for the Kali address and rule 554 at level 5 for the 68-byte EICAR file. These are concrete, repeatable observations rather than expected or simulated dashboard values.

The service-health result also demonstrates the cost of consolidation. Only about 4 GB physical memory was available, and the 2 GB swap allocation was heavily used. The outcome supports the recommendation that production services be separated or allocated additional memory even though the consolidated demonstration remained functional.

5.9 Screenshot Interpretation

Each screenshot is included for a specific evidentiary purpose. MISP screenshots prove event and attribute creation; terminal captures prove Docker deployment and API communication; Kali output proves the source and target of the scan; Wazuh Threat Hunting proves the resulting rule ID and severity; and FIM evidence proves that the observed SHA-256 was generated by Wazuh rather than manually asserted. The captions and surrounding discussion therefore connect each image to an explicit project objective.

6. Analysis and Recommendations

6.1 Interpretation

The prototype validates the central hypothesis. Wazuh provided local behavioural and file-integrity detections, while MISP supplied context about whether the observed IP address and hash were registered IOCs. The same integration code handled two indicator classes by selecting the appropriate field from the Wazuh alert JSON.

The consolidation of MISP and Wazuh on one small server introduced genuine operational challenges: HTTPS port conflict, storage pressure, indexer memory exhaustion, unstable image downloads and the FIM 100,000-file limit. Resolving these issues produced a realistic account of resource planning and troubleshooting in a SOC environment.

6.2 MITRE ATT&CK; Mapping

Observed activity	ATT&CK; mapping	Wazuh evidence
SSH password guessing	T1110.001 — Password Guessing; T1021.004 — SSH	Rules 5710/5760
Repeated failures	T1110 — Brute Force	Rule 2502
Known-bad source connection	Remote Services / IOC correlation	Rule 100002 + MISP match
EICAR file creation	Payload placement / file modification visibility	Rule 554 + SHA-256

6.3 Limitations

- The consolidated server has only about 4 GB RAM; 2 GB swap was required to keep the indexer stable.
- The Wazuh local manager (agent ID 000) acted as the monitored endpoint instead of a separate endpoint VM.
- MISP feed metadata was available, but the deterministic tests used curated lab indicators.
- FIM was temporarily restricted to the EICAR test directory to avoid the stale 100,000-file limit.
- The lab used self-signed certificates and private VMware NAT addresses.

6.4 Recommendations

- Separate MISP and Wazuh onto dedicated hosts for production workloads.
- Allocate at least 8 GB RAM to a consolidated lab or 4–8 GB per dedicated service host.
- Rotate the exposed lab API key and use a limited-permission integration user.
- Restore broader FIM monitoring after tuning file limits and exclusions.

- Add a SOAR platform such as TheHive or Shuffle for automated case creation and containment.
- Automate deployment with Ansible and add a Windows/Sysmon endpoint.

6.5 Comparison with a SIEM-Only Workflow

Activity	SIEM only	Integrated TIP-SIEM
Known-bad IP validation	Analyst manually searches external sources	MISP query occurs automatically from the Wazuh alert
File-hash validation	Hash must be copied to an external portal	syscheck.sha256_after is sent to MISP automatically
Alert context	Rule, source and raw log	Rule plus event ID, indicator type and matching value
Repeatability	Depends on analyst procedure	Encoded in rule IDs and custom integration logic

6.6 Operational Lessons

- Architecture diagrams must represent the implemented network rather than the initial four-VM proposal.
- Resource planning is a security requirement because an OOM-killed indexer removes alert-search capability.
- Secret handling must extend to screenshots and Git history, not only configuration files in the latest commit.
- A successful HTTP response is not sufficient evidence by itself; the returned attribute count and value must also be verified.
- FIM baselines must complete successfully before file-creation tests are interpreted.
- Live demonstrations should have screenshot evidence as a fallback, as required by the presentation guidance.

6.7 Recommended Production Architecture

A production evolution should place MISP, Wazuh Indexer, Wazuh Manager/Dashboard and monitored endpoints on separate systems or resource pools. TLS certificates should be issued by a trusted internal CA, API access should use a dedicated read-only MISP integration user, FIM limits should be tuned against approved directories, and active response should be added only after false-positive testing.

7. Conclusion

This project successfully implemented a working MISP-Wazuh prototype on one Ubuntu Server and validated it using real attack and file events. MISP and Wazuh coexisted through separate HTTPS ports. Kali-originated activity was detected by Wazuh custom rule 100002 at level 12, and the EICAR file was detected in real time through FIM rule 554. The integration automatically queried MISP for both 192.168.18.128 and the EICAR SHA-256, receiving HTTP 200 and one matching attribute for each indicator.

The completed prototype demonstrates that an open-source TIP can add operational context to SIEM detections without paid infrastructure. The recorded challenges and resolutions—storage expansion, resilient image downloads, swap configuration, custom script argument handling and FIM database repair—also provide a practical foundation for future SOC lab work.

8. References

- [1] MISP Project, "MISP — Open Source Threat Intelligence Platform." <https://www.misp-project.org/>
- [2] Wazuh, "Wazuh Documentation." <https://documentation.wazuh.com/>
- [3] abuse.ch, "URLhaus and ThreatFox." <https://abuse.ch/>
- [4] CIRCL, "CIRCL OSINT Feed." <https://www.circl.lu/>
- [5] OASIS, "STIX Version 2.1," 2022.
- [6] OASIS, "TAXII Version 2.1," 2021.
- [7] MITRE, "ATT&CK; Knowledge Base." <https://attack.mitre.org/>
- [8] E. Hutchins, M. Cloppert and R. Amin, "Intelligence-Driven Computer Network Defense," 2011.
- [9] NIST SP 800-150, "Guide to Cyber Threat Information Sharing," 2016.
- [10] FIRST, "Traffic Light Protocol," 2022.
- [11] VMware, "VMware Workstation Documentation."
- [12] Docker, "Docker Engine Documentation." <https://docs.docker.com/>
- [13] EICAR, "Anti-Malware Test File." <https://www.eicar.org/>
- [14] OffSec, "Kali Linux Documentation." <https://www.kali.org/docs/>

9. Appendices

Appendix A — Key Commands

```
# Wazuh installation
sudo bash wazuh-install.sh -a -o -p 8443

# MISP containers
cd ~/misp-docker
sudo docker compose up -d
sudo docker compose ps

# Kali tests
nmap -sS 192.168.18.130
ssh kali@192.168.18.130

# EICAR
curl -L -o /tmp/eicar.com https://secure.eicar.org/eicar.com
sha256sum /tmp/eicar.com
```

Appendix B — Deployment Evidence

```
+ Building 99./s (4/19)                                docker:default
=> [misp-modules internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 2.75kB                                0.0s
=> [misp-modules internal] load metadata for docker.io/library/python:3.14-slim-trixie 3.6s
=> [misp-modules internal] load .dockerignore                        0.0s
=> => transferring context: 2B                                         0.0s
=> [misp-modules python-build 1/11] FROM docker.io/library/python:3.14-slim-trixie@sha256: 75.5s
=> => resolve docker.io/library/python:3.14-slim-trixie@sha256:cea0e6040540fb2b965b6e7fb5ffa 0.0s
=> => sha256:e643ab40c29e592ca9672a5b9b1e47d6845fe849c459585dc999772f60034d18 248B / 248B 0.5s
=> => sha256:5b2fb5800bc41ebae70b49dde01f5b018473be472f0980e152c26e993ce8 12.34MB / 12.34MB 38.8s
=> => sha256:062e450697faa5f02a3a74eba9864ee4d79bc9cfbd65769fc6cdf2c05c6 29.78MB / 29.78MB 73.5s
=> => sha256:e4ef6bf557b964af2625a37b765b3c8890f7a5fca6a4fd8f58d70db1adfe164 1.29MB / 1.29MB 8.0s
=> => extracting sha256:062e450697faa5f02a3a74eba9864ee4d79bc9cfbd65769fc6cdf2c05c6a053 1.2s
=> => extracting sha256:e4ef6bf557b964af2625a37b765b3c8890f7a5fca6a4fd8f58d70db1adfe164e 0.1s
=> => extracting sha256:5b2fb5800bc41ebae70b49dde01f5b018473be472f0980e152c26e993ce86f1c 0.6s
=> => extracting sha256:e643ab40c29e592ca9672a5b9b1e47d6845fe849c459585dc999772f60034d18 0.0s
=> [misp-modules stage-1 2/6] RUN <<-EOF (apt-get update...)      20.5s
=> => # Hit:1 http://deb.debian.org/debian trixie InRelease
=> => # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
=> => # Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
=> => # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9673 kB]
=> [misp-modules python-build 2/11] RUN <<-EOF (apt-get update...) 20.5s
=> => # Hit:1 http://deb.debian.org/debian trixie InRelease
=> => # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
=> => # Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
=> => # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9673 kB]
```

Figure E-1 — Real MISP image build attempt.

```
ibrahim@ubuntuuserver:~/misp-docker$ grep -A2 "misp-modules:" docker-compose.yml | head -10
  misp-modules:
    condition: service_healthy
    healthcheck:
--
  misp-modules:
    image: ${REGISTRY_MIRROR_URL:-}ghcr.io/misp/misp-docker/misp-modules:${MODULES_RUNNING_TAG:-late
st}
    restart: always
    build:
```

Figure E-2 — Real Compose service definition for misp-modules.

Appendix C — Security and Reproducibility Notes

- The lab API key was redacted in this report and must be rotated after assessment.

- The password 1111 was used only for the isolated lab administrator account and is not suitable for production.
- All attack traffic was directed at systems owned and controlled by the student.
- The EICAR file is a harmless industry-standard test string.

Appendix D — GitHub Submission Checklist

- README contains the project title, summary, student identity, tools and reproduction steps.
- Final PDF is stored in report/.
- Custom MISP integration scripts and Wazuh rules are stored in scripts/ and configs/.
- Original evidence is stored in evidence/ with descriptive filenames.
- .gitignore excludes .env, keys, certificates, VM disks and credentials.
- Repository history is reviewed to ensure no API key or password remains in earlier commits.

Appendix E — Presentation Evidence Plan

- Problem and objective: context-less SIEM alerts.
- Setup: consolidated MISP/Wazuh server and Kali attacker.
- Evidence: MISP event, rule 100002, EICAR rule 554 and integration HTTP 200 matches.
- Recommendations: separate production hosts, increase memory, rotate secrets and restore tuned FIM monitoring.
- Fallback: use the real screenshots if a live demo is unavailable.

Appendix F — Custom Integration Logic

The complete operational script is stored in the repository. The following excerpt documents the essential argument parsing and indicator-selection logic used in the working prototype.

```
#!/usr/bin/env python3
import json, sys, requests

alert_file = sys.argv[1]
api_key = sys.argv[2]
hook_url = sys.argv[3]

with open(alert_file, "r", encoding="utf-8") as file:
    alert = json.load(file)

data = alert.get("data", {}) or {}
syscheck = alert.get("syscheck", {}) or {}
value = data.get("srcip") or data.get("dstip")
indicator_source = "ip"

if not value:
    value = syscheck.get("sha256_after")
    indicator_source = "sha256"
```

```
headers = {"Authorization": api_key, "Accept": "application/json"}
response = requests.get(hook_url, headers=headers,
params={"value": value, "returnFormat": "json"},
verify=False, timeout=15)
```

Appendix G — Wazuh Rule and Integration Excerpts

```
<integration>
<name>custom-misp</name>
<hook_url>https://192.168.18.130/attributes/restSearch</hook_url>
<api_key>REDACTED</api_key>
<alert_format>json</alert_format>
<rule_id>2502,5760,5763,5503,5715,100002,554</rule_id>
</integration>

<rule id="100002" level="12">
<match>192.168.18.128</match>
<description>Connection from MISP-registered known-bad IP (Kali)</description>
</rule>
```

Appendix H — Detailed Troubleshooting Commands

```
# Indexer / manager health
systemctl status wazuh-indexer wazuh-manager
journalctl -xeu wazuh-indexer.service

# Resumable package download
wget -c -O wazuh-indexer.deb <package-url>

# Docker recovery
systemctl stop docker docker.socket containerd
rm -rf /var/lib/containerd
mkdir -p /var/lib/containerd
systemctl start containerd docker

# FIM reset
systemctl stop wazuh-manager
rm -f /var/ossec/queue/fim/db/fim.db*
systemctl start wazuh-manager
```

Appendix I — Real Screenshot Index

Figure	Real evidence	Purpose
3-4	Docker hello-world and Compose status	Runtime and service health
5-10	MISP event, feeds and attributes	TIP configuration
11	Redacted custom-misp block	Integration configuration
12-14	Pull, startup and image list	Deployment chronology
15	Kali Nmap scan	Attack source and open services
16	Wazuh rule 100002 events	Known-bad IP detection
17	MISP API match output	IOC correlation
18-19	EICAR terminal and Wazuh rule 554	File/hash validation

Appendix J — README Outline

```
# CY376 TIP-SOC Project

## Student
Ibrahim Abdul Aziz — FCM.41.018.151.23

## Summary
MISP + Wazuh enrichment prototype.

## Architecture
Ubuntu Server 192.168.18.130; Kali 192.168.18.128.

## Reproduction
1. Install Wazuh on 8443.
2. Deploy MISP on 443.
3. Create Event ID 1 and indicators.
4. Install custom-misp wrapper and rule 100002.
5. Run Kali and EICAR tests.

## Evidence
See evidence/ and the final report.
```

Appendix K — Ten-Minute Presentation Outline

Time	Slides / activity	Focus
0:00–1:00	Problem and objectives	Why SIEM alerts need threat context
1:00–2:30	Architecture	Consolidated server, ports and Kali
2:30–4:00	Implementation	MISP, Wazuh and custom integration
4:00–6:00	Known-bad IP result	Rule 100002 and MISP ip-dst match
6:00–8:00	EICAR result	Rule 554 and MISP SHA-256 match
8:00–9:15	Challenges	Downloads, OOM, FIM limit and resolutions
9:15–10:00	Recommendations and conclusion	Production improvements and project value